DFMAcc: Geometry-aware FPGA Acceleration for DFM Pattern Matching with Dynamic Layout Partition

ABSTRACT

At advanced process nodes, physical verification has become a critical step in semiconductor design. Among sign-off checks, design-for-manufacturability (DFM) *pattern matching* is essential for detecting lithography- and yield-critical geometries. However, as layout data scales exponentially and pattern libraries reach tens of thousands, conventional CPU/GPU frameworks suffer from sparse candidates, irregular data access, and frequent data movement, creating severe runtime bottlenecks. This paper presents **DFMAcc**, a geometry-aware FPGA acceleration for DFM *pattern matching*. **DFMAcc** establishes a full streaming pipeline from region dispatch to geometric verification. On the software side, a pattern-aware adaptive partitioning algorithm balances task granularity and boundary integrity based on candidate density and geometric complexity. On the hardware side, a geometry-aware hash matching engine performs early strong candidate filtering through orientation encoding and two-level verification, maintaining stable throughput via FIFO decoupling and multi-engine/multi-channel organization. The number of candidates entering geometric verification is reduced by over 95% on average, achieving robust scalability for large-scale sign-off DFM *pattern matching*. Evaluations on the AMD Alveo U280 prototype demonstrate robust throughput, achieving up to 124× speedup over the industrial Calibre baseline. Furthermore, **DFMAcc** successfully processes an authentic hundred-billion-scale commercial SRAM layout in ~8.3 hours, demonstrating its extreme scalability and readiness for industrial sign-off.

1 INTRODUCTION

As semiconductor manufacturing scales to advanced process nodes, the interplay between layout geometries and lithography processes has become increasingly complex. Physical verification (PV), formerly a routine validation of geometric rules, has evolved to incorporate sophisticated Design-for-Manufacturing (DFM) analysis to ensure yield and printability. Meanwhile, traditional rule-based Design Rule Checking (DRC) often fails to capture the complex, non-local geometric interactions that precipitate lithography hotspots. To address this limitation, *pattern matching* has emerged as a critical complementary sign-off methodology, enabling the precise identification of yield-detracting structures by matching layouts with extensive libraries of known problematic topologies [1, 2].

Despite its significance, *pattern matching* imposes substantial computational complexity, posing a challenge for industrial-scale applications. Modern industrial tools, such as Calibre Pattern Matching [3, 4], are required to process expansive full-chip layouts against tens of thousands of variant-tolerant templates. This intensive computational load has rendered runtime the primary bottleneck in achieving design closure. As illustrated in Fig. 1, an analysis of runtime transcripts from five industrial GDSII designs on the Calibre baseline confirms this bottleneck: the match phase alone accounts for over 60% of the total runtime, while partitioning contributes an additional 10–20%.

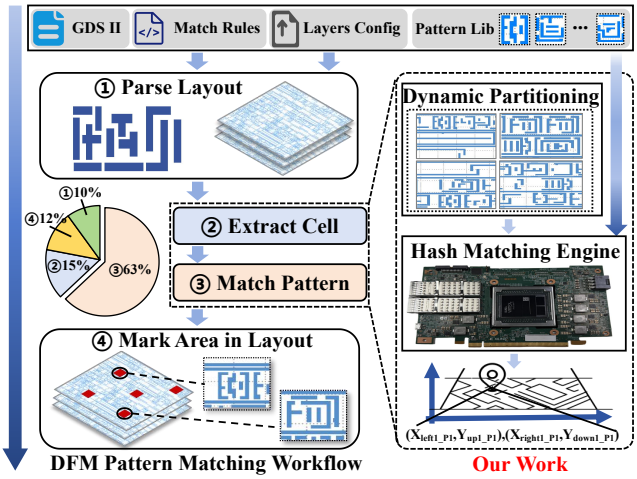


Figure 1: Comparison between the DFM pattern matching flow and our DFMAcc architecture.

Existing software and hardware acceleration frameworks for DFM *pattern matching* exhibit limited scalability. CPU-based engines typically adopt spatial decomposition for workload distribution; however, they frequently encounter severe load imbalances due to the non-uniform distribution of critical patterns in real-world designs. Conversely, GPU-based acceleration encounters a fundamental architectural mismatch. While GPUs excel at structured, rule-based tasks like DRC [5–8] or dense optimizations like placement [9], *pattern matching* is characterized by a sparse, irregular, and template-dependent workload. This divergence from regular computation creates a worst-case scenario for the GPU's SIMT execution model, causing severe control divergence and inefficient memory coalescing that prevent the realization of theoretical peak performance. Furthermore, previous FPGA research has focused on structured problems like routing acceleration [10], leaving a gap in architectural support for the irregular geometric structures required by *pattern matching*.

The aforementioned limitations stem from a fundamental mismatch between the intrinsic data-dependent characteristics of *pattern matching* and the constraints of traditional batch-parallel architectures. Unlike grid-based simulations[11], *pattern matching* is inherently a streaming problem, *i.e.*, each candidate region is spatially independent but requires a serialized sequence of geometric checks. This execution model aligns naturally with spatial-flow FPGA architectures, where deeply reconfigurable pipelines can process geometric streams with deterministic latency, thereby circumventing the severe synchronization penalties inherent to CPUs and GPUs. Furthermore, FPGAs facilitate the construction of custom memory hierarchies tailored to the sparse access patterns of geometric data, maximizing effective bandwidth.

In this paper, we present **DFMAcc**, a geometry-aware streaming FPGA architecture designed for extreme-scale industrial DFM *pattern matching*. To demonstrate its practical efficacy, **DFMAcc** has

been successfully deployed against an authentic hundred-billion-scale commercial layout. Rather than merely scaling hardware resources, **DFMAcc** addresses geometric irregularity through a hardware-software co-design approach. In software, we propose a frequency-based anchor extraction to reduce the initial search space and apply dynamic layout partitioning to prevent processing hotspots. In hardware, **DFMAcc** maps the remaining matching tasks into a two-stage streaming pipeline, *i.e.*, Hash Filtering and Geometric Verification, to maintain peak memory bandwidth utilization. This full-stack integration provides a scalable path for *pattern matching* acceleration, redefining the standard verification flow. The main contributions of this paper are as follows.

- To the best of our knowledge, **DFMAcc** is the first dedicated hardware architecture for DFM *pattern matching* that fully leverages hardware-software synergy. The proposed design establishes a geometry-aware streaming pipeline that bridges the proposed software pre-processing with deterministic hardware geometric verification.
- In software, we introduce a frequency-based anchor extraction to prune the expansive initial search space. This is coupled with a density-driven dynamic partitioning algorithm that ensures balanced workload distribution while maintaining strict boundary integrity across layout regions.
- In hardware, we design a fully pipelined, two-stage FPGA engine that decouples orientation-encoded hash filtering from fine-grained geometric verification. Featuring specialized subtractor trees and a dynamic mode-flag architecture, the engine natively supports both exact and fuzzy matching while maintaining peak line-rate streaming throughput.

Evaluations on an AMD Alveo U280 prototype demonstrate up to **124**× speedup over the industrial Calibre baseline. Furthermore, to push architectural limits and demonstrate industrial readiness, **DFMAcc** was evaluated on an authentic hundred-billion-scale ($> 10^{11}$ polygons) commercial full-chip SRAM layout under strict non-disclosure agreements (NDAs). By processing this colossal industrial dataset in merely ~8.3 hours, **DFMAcc** achieved a 6.00× speedup over the 32-core Calibre baseline, confirming that geometry-aware streaming computation offers a highly viable and scalable path for extreme-scale DFM sign-off.

2 BACKGROUND AND MOTIVATION

2.1 Pattern Matching Workflow

DFM *pattern matching* is a key component in modern physical verification, complementing rule-based DRC by detecting complex geometries that affect lithography printability and yield [1, 2, 12, 13]. Commercial sign-off tools such as Calibre Pattern Matching [3, 4] execute these checks across terabyte-scale layouts against thousands of pre-characterized templates. As shown in Fig. 2, a representative industrial *pattern matching* flow typically includes four stages:

(1) Layout parsing: The polygon information within the layout is first extracted. Polygons on the target pattern layer are then merged and simplified to reduce the computational overhead for downstream operations.

(2) Region partitioning: The large-scale layout is divided into fixed-size tiles (small blocks) to facilitate massive parallelization. After partitioning, each tile boundary is appropriately expanded

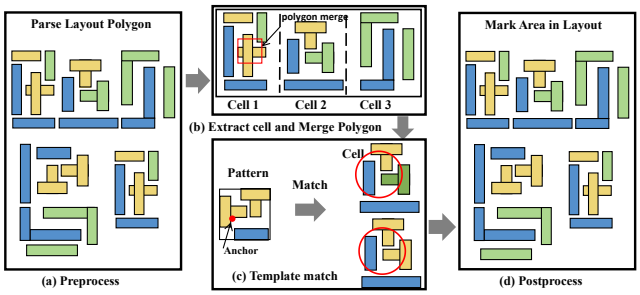


Figure 2: DFM pattern matching flow.

to prevent pattern truncation at the boundaries and ensure that structures spanning multiple blocks are correctly identified.

(3) Template matching: A scanline algorithm traverses the geometric primitives within each tile. Anchor points, specific geometric vertices used as reference coordinates, are dynamically extracted and correlated against the pattern library to identify potential matches within the layout tiles.

(4) Post-processing: Identified matches undergo an affine transformation to map local tile coordinates back to the global layout. The final results are aggregated and exported for yield analysis.

As layout complexity and library sizes scale, these stages have emerged as the primary bottlenecks threatening verification throughput and tape-out predictability. DFM *pattern matching* poses unique acceleration challenges due to its geometry-driven and highly irregular nature. Hierarchical layouts, multi-layer interactions, and uneven polygon density cause severe spatial imbalance; dense areas create computation hotspots, while sparse regions lead to underutilization. Moreover, pattern templates involve arbitrary orientations and layer combinations, producing sparse and irregular data access. At the same time, industrial sign-off requires deterministic, bit-exact results. This requirement differentiates sign-off *pattern matching* from non-deterministic or learning-based EDA tasks, such as ML-driven hotspot detection or capacitance extraction [14–18].

2.2 Motivation

The irregular and geometry-dependent nature of DFM *pattern matching* creates a significant structural mismatch with traditional parallel hardware. The process involves sequential stages of feature filtering and fine-grained geometric validation for independent regions, forming a streaming pattern that demands low-latency execution. Conventional CPU/GPU platforms struggle with this highly irregular workload due to inescapable synchronization penalties and severe thread divergence. Even advanced software-level geometric optimizations [19] cannot fully overcome the inherent hardware bottlenecks of fixed-instruction architectures.

FPGAs offer a compelling architectural alternative by providing an execution model that perfectly aligns with the streaming requirements of *pattern matching*. By leveraging deeply reconfigurable pipelines and specialized memory hierarchies, FPGAs can process irregular geometric data with strict deterministic timing and minimal overhead. The ability to deploy fine-grained spatial engines that maintain stable throughput despite varying layout densities makes FPGAs the ideal platform for this task. These advantages drive our design of a geometry-aware streaming architecture to achieve a scalable, industrial-grade acceleration flow.

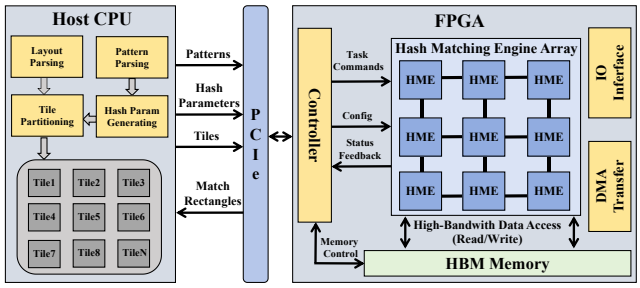


Figure 3: Overall architecture and dataflow of DFMAcc.

3 OVERVIEW

As shown in Fig. 3, **DFMAcc** is an end-to-end heterogeneous acceleration framework designed for scalable DFM *pattern matching*. To overcome severe load imbalance, uncoalesced memory access, and irregular data access bottlenecks in conventional CPU/GPU platforms, **DFMAcc** transforms the highly irregular *pattern matching* process into a deterministic streaming dataflow. The framework is achieved through a hardware–software co-design paradigm comprising two tightly integrated components:

(1) Dynamic Layout Partitioning. The host CPU is responsible for global data preparation and workload orchestration. Instead of using a uniform grid-based decomposition, a *Pattern-Aware Dynamic Partitioning* algorithm (detailed in Section 4) first analyzes local geometric complexity and polygon density of the parsed layout. Based on this analysis, the partition granularity is adaptively adjusted. A boundary integrity verification mechanism is further applied to resolve cross-region spatial dependencies. This preprocessing stage mitigates computational hotspots and regularizes data distribution, ensuring that generated sub-regions are both geometrically independent and well-balanced. Concurrently, the host performs an additional preprocessing step to optimize downstream hardware matching. By statistically analyzing pattern occurrences against the layout histogram, the software identifies the least frequent edge as a global matching anchor, significantly reducing the hardware search space prior to execution.

(2) FPGA-based Hash Matching Engine. The partitioned layout edges and pattern hash templates are streamed to the FPGA through a high-bandwidth PCIe interface. To efficiently process this data stream without incurring control-flow divergence, the FPGA implements a Geometry-Aware Hash Matching Engine (detailed in Section 5). Leveraging High-Bandwidth Memory (HBM) for scalable storage of large pattern libraries and enabling massive parallel data access, the hardware adopts a fully pipelined two-stage architecture. *Stage 1 (Candidate Filtering):* A hash calculator array rapidly filters out invalid candidates using orientation-encoded signatures, inherently supporting multi-layer parallel verification. A deep candidate FIFO buffer decouples this stage from downstream processing, absorbing bursty traffic and maintaining stable throughput. *Stage 2 (Matching Verification):* Geometric verification is performed exclusively on high-confidence candidates, supporting both exact- and fuzzy-matching modes. The final matched coordinates are transmitted back to the host via overlapped DMA transfers.

By integrating adaptive layout partitioning with a dedicated streaming pipeline, **DFMAcc** establishes a balanced dataflow. This architecture eliminates conventional control-flow overheads and enables near-linear scalability for industrial-scale DFM verification.

Algorithm 1: Adaptive Block Size Calculation

Input: δ, A_q, N_p, ρ
Output: Block size B

1 $A_{\min} \leftarrow \sqrt{\alpha_{\min}\alpha_{\max}}A_t, \quad A_{\max} \leftarrow \alpha_{\max}A_t$;
2 $H_{\min} \leftarrow \lfloor A_t/(\kappa_H A_{\min})\rfloor, \quad H_{\max} \leftarrow \lfloor A_t/(\kappa_L A_{\max})\rfloor$;
3 **if** $N_p \le H_{\min}$
4 $\quad A_{tgt} \leftarrow A_{\min} \times \left(1 + \gamma_H \frac{N_p - H_{\min}}{N_p}\right)$;
5 **else if** $N_p \ge H_{\max}$
6 $\quad A_{tgt} \leftarrow A_{\max} \times \left(1 - \gamma_L \frac{H_{\max} - N_p}{H_{\max}}\right)$;
7 **else**
8 $\quad A_{tgt} \leftarrow A_{\min} + (A_{\max} - A_{\min}) \frac{H_{\min} - N_p}{H_{\min} - H_{\max}}$;
9 $\omega_{load} \leftarrow \max(\varepsilon_m,\ \min(1 + \lambda\rho - \delta,\ \varepsilon_M))$;
10 $\omega_{var} \leftarrow 1 + \beta_c\sqrt{\rho}$;
11 $\omega \leftarrow \omega_{load} \times \omega_{var}$;
12 $B_{raw} \leftarrow \text{clip}(A_{tgt} \times \omega,\ A_{\min},\ A_{\max})$;
13 $B \leftarrow \lfloor B_{raw}/A_q \rfloor \times A_q$;
14 **if** $A_t/B > N_{\max}$ $B \leftarrow \lfloor A_t/N_{\max} \rfloor$;
15 **return** B;

4 LAYOUT DYNAMIC PARTITIONING

Efficient region partitioning is critical for maximizing the throughput of the proposed streaming architecture. A naive uniform grid approach cannot accommodate the extreme spatial variability of layouts: dense areas cause computational hotspots, whereas sparse regions incur redundant scheduling overhead. This workload imbalance directly degrades hardware efficiency. To address this, we propose a pattern-aware adaptive partitioning framework. By dynamically adjusting partition granularity based on local geometric complexity, we can achieve a more balanced workload distribution and a continuous data supply to the FPGA engines.

4.1 Geometric Complexity Modeling

The adaptive partitioning process relies on a lightweight geometric model that estimates the local workload and matching difficulty. This model leverages three statistical metrics:

(1) Polygon count (N_p) measures the number of polygons within a region and directly reflects the computation scale.

(2) Normalized density ($\rho \in [0, 1]$) captures the degree of geometric aggregation within a region, defined as the ratio of total polygon area to the bounding box area.

(3) Density variance (δ) quantifies spatial non-uniformity of geometric distribution by dividing each region into subgrids and computing the variance of subgrid densities.

Based on these metrics, Algorithm 1 acts as a spatial equalizer, dynamically determining the optimal block size B under the Density Compensation Principle. Avoiding a naive static grid's load imbalance, it evaluates local geometry to prevent hotspots and underutilization, first establishing bounds $[A_{\min}, A_{\max}]$ and thresholds $H_{\min}$, $H_{\max}$ (Lines 1–2).

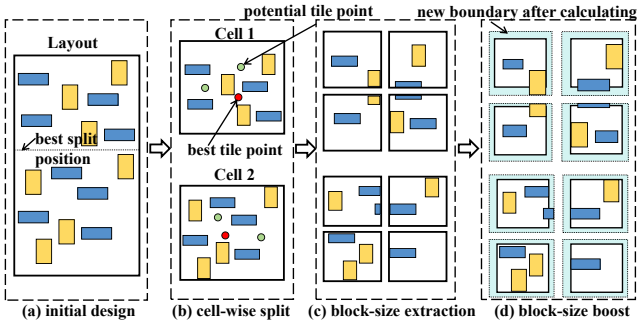


Figure 4: Dynamic partitioning flow.

To effectively suppress hotspots and reduce redundant scheduling overhead, the algorithm computes a preliminary target area A_{tgt} via piecewise conditional scaling based on the polygon count N_p (Lines 3-8). For sparse zones ($N_p \leq H_{\min}$), it expands the block size to bundle dispersed geometries, thereby minimizing the fixed latency overhead associated with region scheduling. For high-density regions ($N_p \geq H_{\max}$), it aggressively shrinks the block size to distribute the intensive workload across multiple hardware engines. Conversely, For medium-density cases, a linear interpolation is applied to ensure a smooth transition between the area bounds.

This preliminary A_{tgt} is subsequently modulated by a hardware-constrained combined factor ω (Line 9-11). Within ω, the sensitivity hyperparameter λ dynamically relaxes the block size when the structural variance δ is prominent, effectively flattening the computational load across unevenly distributed patterns. Concurrently, the penalty weight β_c strictly prevents downstream BRAM overflow in highly clustered regions by mathematically compensating for the non-linear matching overhead caused by extreme polygon aggregation. Finally, the modulated product is rigorously clipped to the predefined geometric bounds, quantized to align with the partition grid resolution A_q, and validated against the maximum block count limit $N_{\max}$ to satisfy host memory constraints (Lines 12-13). This lightweight yet robust heuristic ensures that the generated sub-regions optimally balance the workload distribution and memory feasibility across the parallel FPGA engines.

4.2 Dynamic Programming Solution

Building on the adaptive block-size model and the geometric complexity metrics introduced in Section 4.1, the global partitioning problem is formulated as a Dynamic Programming (DP) optimization. As illustrated in Fig. 4 (a)-(c), unlike static uniform slicing, the proposed method recursively explores candidate split positions to identify an optimal decomposition strategy. Rather than relying on a generic cost function, the DP evaluates the cost of each candidate sub-region directly using the previously defined statistical metrics: *polygon count (N_p)*, *normalized density (ρ)*, and *density variance (δ)*. The objective of the DP recursion is to find a set of cuts that minimizes the variance of these costs across all final partitions.

To ensure computational efficiency, the DP recursion explores candidate splits and records the minimal cumulative cost in a path-cost matrix. By storing these intermediate results, each new subdivision decision can directly reuse previously optimized subregions, thereby guaranteeing global rather than local optimality without redundant re-evaluation. Once the optimal configuration is identified, the corresponding sequence of partition boundaries is reconstructed from the matrix. The resulting regions are further refined to preserve polygon integrity and maintain cross-region consistency. This hierarchical optimization strategy enables balanced workload distribution across the entire layout, while minimizing partitioning overhead. As a result, it ensures a continuous and stable data supply to the FPGA engines.

4.3 Boundary Integrity Guarantee

After dynamic partitioning, geometric consistency along region borders must be preserved, as illustrated in Fig. 4 (d), to ensure accurate matching results. Without proper handling, polygons that cross partition boundaries may be duplicated or split, leading to redundant computation and data inconsistencies. To address this issue, a boundary-integrity mechanism is applied following DP optimization. A spatial conflict detector identifies polygons that intersect partition edges, and each conflicted shape is assigned to its dominant region based on geometric overlap. The assignment strategy prioritizes the region containing the polygon's centroid to minimize perimeter fragmentation, breaking ties by lexicographical region ID ordering. This guarantees reproducible partitioning across runs. Adjacent boundaries are then slightly adjusted to maintain topological continuity while keeping region independence. This post-adjustment ensures one-to-one polygon ownership and eliminates cross-region communication. Consequently, all partitions remain geometrically complete and stream-aligned, providing clean, non-overlapping input data to the FPGA matching engines.

4.4 Pre-processing for Pattern Matching

To accommodate tolerance constraints within the pattern library, the host software executes a critical pre-processing stage prior to FPGA offloading. The system first parses the geometric patterns to extract matching constraints. To support both exact matching and fuzzy matching, these constraints are categorized into two types: single-layer floating constraints, which allow positional deviation of a single edge, and two-layer spacing constraints, which enforce distance ranges between parallel edges. The extracted parameters are packaged into lightweight descriptors, enabling the hardware engine to switch between exact- and fuzzy-matching modes.

A cornerstone of this pre-processing stage is the dynamic anchor selection strategy. Rather than naively assigning a static vertex, *e.g.*, the top-left corner, as the reference point for hash computation, the host executes a frequency-aware algorithm. The software analyzes the layout histogram and statistically identifies the pattern edge with the lowest occurrence frequency to serve as the global matching anchor. The underlying rationale is to minimize the number of candidate matches at the earliest stage. For example, given a pattern with edges A, B, and C, if edge A appears 1,000 times in the layout while edge C appears only 5 times, selecting A as the anchor would generate 1,000 candidate starting points requiring full verification. In contrast, selecting edge C reduces the candidate set to only 5. Conversely, selecting the minimum-frequency edge C as the anchor intrinsically restricts the search space to merely 5 candidates. This software-driven pruning drastically minimizes the downstream workload, ensuring that the hardware engine operates at peak efficiency.

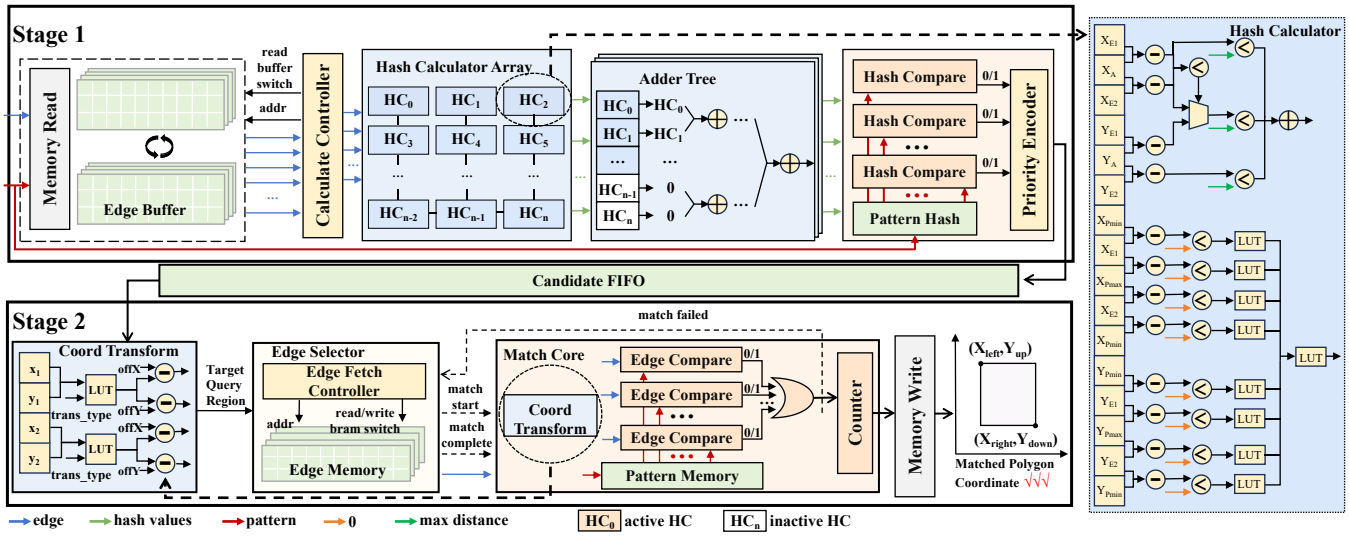


Figure 5: Hash matching engine. The engine consists of two stages: candidate filtering and matching verification.

5 GEOMETRY HASH MATCHING ENGINE

Based on the results of the software pre-processing stage, the proposed geometry-aware matching engine adopts a high-throughput, two-stage streaming FPGA architecture. This design fundamentally decouples the irregular workload into a fast hash-based filtering stage and a subsequent fine-grained geometric verification stage. As shown in Fig. 5, by leveraging orientation-encoded signatures to rapidly narrow the candidate search space, the fully pipelined structure ensures that the computationally expensive verification logic is activated exclusively for high-confidence candidates, thereby maximizing overall hardware resource utilization.

5.1 Candidate Pattern Filtering

The proposed candidate pattern filtering method encodes the geometric features of edges into compact hash signatures for fast prescreening. As shown in Fig. 6, each pattern consists of an *anchor polygon* and its neighboring edges, which are grouped into direction buckets (*right*, *down*, and *right–down*). During hash computation, instead of a static vertex, the hash cuses the precomputed minimum-frequency edge as the reference anchor. The offsets of each edge endpoint relative to this point are calculated: for horizontal edges, Δx_1, Δx_2, and Δy; for vertical edges, Δy_1, Δy_2, and Δx.

During matching, layout polygons compute their hash signatures similarly and compare them with the pattern library. Since hash computation depends on relative geometry to the anchor point, the overall hash signature remains invariant under rotation, mirroring, and translation. To support rotation and mirroring, the hash buckets are extended to four groups of directions: *left–up*, *left–down*, *right–up*, and *right–down*. Transformations between directions are realized by simple index permutations, *e.g.*, a 90° rotation maps (*right*, *down*, *right–down*) to (*down*, *left*, *left–down*).

This method preserves a compact search space while supporting multi-directional matching, efficiently filtering out invalid candidates and providing high-confidence input to downstream hardware pipelines. Crucially, by abstracting complex geometries into compact bit-vectors, this encoding scheme decouples the filtering latency from the exponentially growing pattern library size.

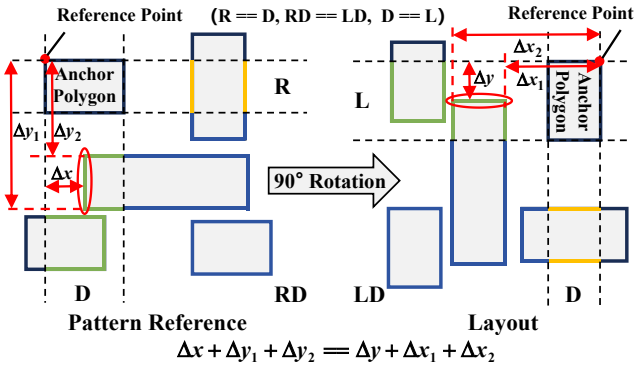


Figure 6: Orientation mapping of polygon hash buckets.

5.2 Hardware Implementation

Engine Overview. As illustrated in Fig. 5, **DFMAcc**'s geometry-aware hash matching engine is designed as a high-performance, fully-streamed dataflow system on FPGA. To handle the exponential growth of layout data, we leverage High Bandwidth Memory (HBM) to store massive polygon edges and industrial-scale pattern libraries, *e.g.*, over 10^5 rules. Layout data and multi-layer pattern features are striped across multiple HBM channels to maximize concurrent memory access. The architecture employs AXI-Stream interfaces for high-speed intra-module communication, with multi-level FIFOs inserted between stages to buffer transient data bursts and decouple the pipeline stages. On-chip ping–pong BRAMs are utilized to overlap the data loading from HBM with the core computation, effectively masking I/O latency and ensuring a deterministic, high-throughput execution environment independent of traditional cache-miss penalties.

Stage 1 – Candidate Filtering. The candidate filtering stage is designed to drastically prune the search space by identifying high-probability regions. The loader module dynamically fetches edge streams based on the specific layers required by the current pattern set, *e.g.*, Poly, Metal, or Via. To balance hardware area and throughput, **DFMAcc** employs a *reference layer selection strategy*, where

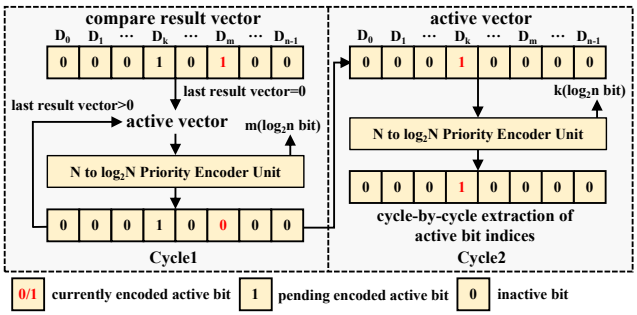


Figure 7: Priority encoder–based match extraction logic.

a single dominant layer is ingested into double-buffered BRAMs for initial hashing. Simultaneously, a global index BRAM tracks geometric metadata, including edge counts, local bounding boxes, and memory base addresses, for every polygon in the reference layer. A parallel hash calculator array, coupled with adder trees, computes orientation-encoded signatures that are invariant to 90-degree rotations and reflections. These signatures are matched in real-time against the pattern library using a parallel comparator bank optimized for wide-vector bitwise operations. This ensures that scaling the pattern library size does not degrade the deterministic streaming throughput, overcoming a major limitation of sequential software matching. A high-speed priority encoder (Fig. 7) then extracts valid hits and generates Candidate Descriptors, which contain the polygon ID, the specific hash match type, and transformation hints. By offloading 94% of non-matching candidates at this stage, the engine significantly reduces the downstream pressure on the more compute-intensive geometric verification.

Stage 2 – Matching Verification. This stage performs rigorous geometric verification to ensure *cross-layer consistency* for the filtered candidates. For each candidate, the engine retrieves geometry data from all relevant layers in accordance with the PDK requirements. Leveraging the transformation hints from Stage 1, the canonical bounding box of the pattern is mapped into the layout coordinate space. To sustain high throughput under varying layout densities, the verification BRAMs are organized into a K-path switching structure, allowing the engine to process K independent candidates in a round-robin fashion, thereby hiding the latency of individual edge fetches. Furthermore, each BRAM is logically partitioned into N segments, each augmented with rectangular boundary metadata ($\langle x_{\min}, y_{\min}, x_{\max}, y_{\max} \rangle$), as shown in Fig. 8. A dedicated *segment-level quick-rejection* unit performs rapid intersection tests, enabling the engine to bypass irrelevant segments and only activate BRAM banks containing potential matching edges.

The active edges are streamed into the *Match Core*, where they undergo parallel coordinate-wise comparison against the pattern template across multiple layers. Driven by a mode flag parsed from the pattern descriptor, the Match Core natively supports both exact and fuzzy geometric verifications. Under the Exact Match Mode, the hardware executes rigid equality checks ($x_{layout} == x_{pattern}$). When toggled to Fuzzy Match Mode to accommodate process variations, the comparator logic evaluates 1-layer floating and 2-layer spacing constraints. Specifically, subtractors compute the relative distances to the anchor and verify them against predefined tolerance intervals ($[\Delta d_{\min}, \Delta d_{\max}]$). This native hardware support

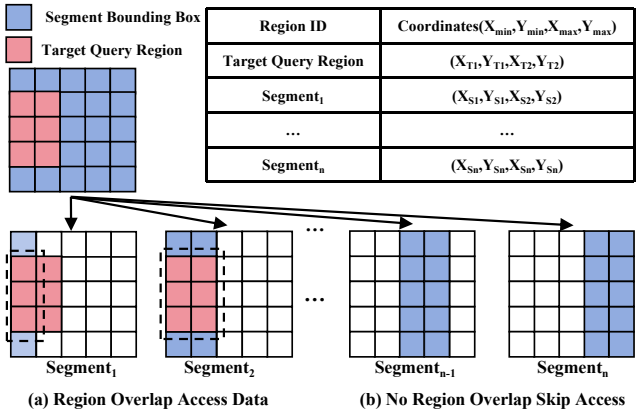


Figure 8: Architecture of the multi-layer K-path BRAM and segment-level filtering.

for multi-layer and variation-tolerant geometric intersection effectively neutralizes the runtime penalties typically associated with advanced multi-patterning nodes. The engine implements an early-termination logic: as soon as a mandatory edge fails the comparison on any layer, the matching slot is immediately released. This combination of K-path interleaving and segment-level pruning ensures that **DFMAcc** maintains peak effective bandwidth even when dealing with highly irregular or dense layout patterns.

6 EVALUATION

6.1 Experimental Setup

Experiments were conducted on three platforms to evaluate **DFMAcc** against industrial and GPU baselines. The CPU baseline uses the Mentor Calibre Pattern Matching tool (running with 32 cores) on an AMD EPYC 9754 128-core processor with 1.5 TB memory. The GPU baseline adopts an NVIDIA A100 accelerator utilizing a highly optimized, synchronization-free CUDA kernel. To ensure comprehensive evaluation, the reported GPU execution times explicitly include all host-to-device (PCIe) data transfer overheads. The FPGA prototype is implemented on a Xilinx Alveo U280 card operating at 150 MHz, equipped with 8 GB HBM and a PCIe Gen4 ×16 interface. Design synthesis and implementation were performed using Vivado 2023.1. The post-implementation on-chip power consumption is reported as 41.04 W, and the detailed hardware resource utilization is summarized in Table 1.

All tests were run on CentOS 7.9, with end-to-end runtime that includes layout parsing, partitioning, matching, and result aggregation. Seven representative layouts across mainstream nodes (12 nm to 28 nm) were evaluated, covering polygon counts from 1.4×10^5 to 4.2×10^{11}. To evaluate the system without relying solely on proprietary foundry data, we synthesized a massive universal library of over 100,000 unique rules by systematically extracting topological windows from open-source GDSII layouts. Emulating authentic sign-off procedures, the software pre-processor activates a design-specific subset of 10,000 patterns for Cases 1-5, and 10 critical hotspots for the NDA-restricted industrial Cases 6-7. These subsets are consistently used across all platforms to ensure fair benchmarking. Finally, all FPGA-generated results were cross-validated against Calibre to guarantee 100% bit-exact sign-off compatibility.

Table 1: Hardware resource utilization.

Resource	Utilization	Available	Percent (%)
LUT	599354	1303680	46%
FF	759408	2607360	27%
LUTRAM	157596	600960	26%
BRAM	1622	2016	80%
URAM	732	960	76%

6.2 System-Level Performance Evaluation

Table 2 summarizes the end-to-end runtime comparison among Calibre, GPU, and FPGA implementations. Across all benchmarks, **DFMAcc** consistently outperforms both software and GPU baselines. For compact designs (Case 1, 141K polygons), the FPGA achieves a 110.00× speedup over Calibre and 4.70× over GPU, validating the baseline efficiency of the streaming architecture. As layout complexity scales to 9.6M polygons (Case 5), the system sustains robust performance, delivering a 95.20× speedup over the CPU baseline and effectively reducing the execution time from over five days (455,570.10 s) to just 1.3 hours (4,785.40 s). Notably, the peak acceleration reaches 124.00× on Case 4, demonstrating the architecture's strong compute capability and resilience to varying geometric densities.

DFMAcc was further deployed against two authentic commercial full-chip layouts provided by our industrial partners: a hundred-billion-scale ($> 10^{11}$ polygons) 28nm SRAM and a ten-million-scale ($> 10^7$ polygons) 12nm NPU Logic core (Cases 6* and 7*). Although strict commercial non-disclosure agreements (NDAs) restricted the evaluation to 10 critical hot-spot patterns, the streaming architecture gracefully processed the complex NPU Logic block in less than 3 minutes and the colossal SRAM in ~8.3 hours. This represents a substantial acceleration over the 32-core Calibre baseline, which required ~4.1 hours and ~50 hours, respectively. The moderation in Case 6's speedup (6.00×) reflects a natural shift in execution bottlenecks: as the hardware accelerates core matching, the overall runtime is bounded by necessary data movement and host-side parsing at this extreme scale. Nonetheless, reducing turnaround time from days to hours demonstrates the practical viability of **DFMAcc** for industrial sign-off procedures.

Compared with GPU and CPU, **DFMAcc** maintains robust performance across varying geometric densities. The streaming pipeline eliminates host–device stalls and redundant memory transfers, while the pattern-aware partitioning balances task distribution across regions. Unlike GPUs that exhibit performance saturation beyond 10^6 polygons, the FPGA sustains high execution efficiency with design scale. Overall, **DFMAcc** achieves up to **124× speedup** over industrial Calibre, confirming that geometry-aware streaming computation provides a scalable and practical acceleration path for large-scale DFM *pattern matching*.

6.3 Hash Matching Engine Efficiency

Pipeline analysis. The end-to-end speedup observed in Table 2 is driven by the Hash Matching Engine's efficiency, quantified in Table 3. Candidate Filtering (Stage 1) filters the initial polygon stream, reducing candidates passed to the matching verification

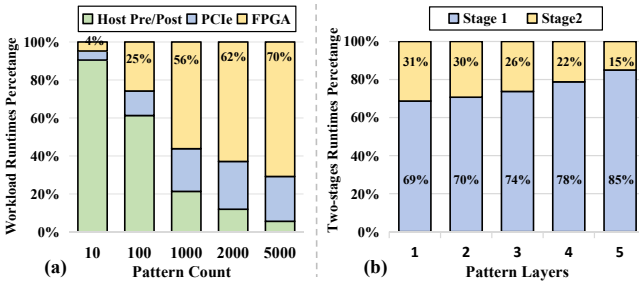


Figure 9: Runtime bottlenecks and pipeline balance.

(Stage 2) by an average of 95.2%. Notably, because the orientation-encoded hash signatures rely on topological features rather than strict distance constraints, this massive candidate reduction is identical and equally effective for both exact and fuzzy matching modes. The distinction between exact and tolerance-based verification is entirely deferred to Stage 2, ensuring that the computational overhead of fuzzy matching is applied only to a highly condensed set of high-confidence candidates.

Fig. 9(a) presents an Amdahl's Law analysis (Case 3) to illustrate performance bottlenecks under varying pattern library sizes (N_{pat}). When the workload is minimal ($N_{pat} = 10$), the fixed-cost Host Pre/Post-processing dominates, consuming approximately 90% of the total runtime. However, as the workload scales to the industrial level ($N_{pat} = 10{,}000$), this static host cost is rapidly amortized to under 3%. The bottleneck then firmly shifts to the compute-bound hardware stages (PCIe Transfer and On-Chip Compute, occupying over 97% of runtime), effectively demonstrating that scaling our FPGA streaming architecture yields direct, near-linear reductions in end-to-end latency.

Fig. 9(b) delves into the efficiency of the on-chip pipeline itself, using data from the 10,000-pattern benchmark runs. It shows the normalized time balance between the two key stages. Stage 1: Candidate Filtering effectively filters the candidate stream, consuming 60-75% of on-chip time, reflecting its intensive filtering effort. This enables Stage 2: Matching Verification to operate efficiently on a significantly smaller dataset, demonstrating the balance of our pipeline design.

Hash Matching Engine Performance Fig. 10(a) compares the matching engine's performance across CPU, GPU, and FPGA. The FPGA matching engine demonstrates significant, stable speedup over both software-centric baselines. It achieves an average of 140× acceleration over the baseline CPU and 6.5× over the A100 GPU, peaking at 165× over the CPU for Case 1. While GPUs excel at dense, regular matrix computations, the irregular, geometry-dependent nature of *pattern matching* inherently limits SIMT efficiency. In contrast, the FPGA matching engine naturally accommodates this sparsity by sustaining continuous execution via deeply pipelined, lock-free dataflows. By transforming geometry verifications into a deterministic spatial pipeline, the FPGA maintains robust acceleration and deterministic latency regardless of layout variations.

Fuzzy Match Constraint Scalability. Beyond exact geometry matching, DFMAcc also supports complex fuzzy DFM rules (e.g., tolerances on area and spacing). As quantified in Table 4, we evaluated scalability using a 100-pattern subset on Case 3, scaling the multi-dimensional fuzzy constraints from 0 to 8. For the CPU baseline, introducing constraints initially reduces total runtime. This occurs

Table 2: End-to-end runtime comparison across platforms.

Design	Name	Node	Polygons	CPU (s)	GPU (s)	FPGA (s)	FPGA vs. CPU	FPGA vs. GPU
Case 1	AES-128	28 nm	141,867	34540.50	1475.80	314.01	110.00×	4.70×
Case 2	UCT-Core	28 nm	183,320	18618.90	1083.60	215.00	86.60×	5.04×
Case 3	OpenC910-SRAM	28 nm	268,248	15226.20	845.30	165.10	92.20×	5.12×
Case 4	OpenC910-IO	28 nm	6,058,254	105400.00	4462.50	850.00	124.00×	5.25×
Case 5	OpenC910-SoC	28 nm	9,633,057	455570.10	36321.20	4785.40	95.20×	7.59×
Case 6*	SRAM2	28 nm	4.2×10^{11}	180125.40	112578.30	30020.90	6.00×	3.75×
Case 7*	NPU-Logic	12 nm	10,327,826	15025.60	1151.20	170.30	88.23×	6.76×

* Evaluated under strict commercial non-disclosure agreements (NDAs) with our industrial partners, restricting the testbench to 10 critical hot-spot patterns.

Table 3: Candidate filtering by match engine.

Design	Polygons	Stage1	Stage2	Filtering (%)
Case 1	141,867	134,773	7,094	95%
Case 2	183,320	175,998	7,322	96%
Case 3	268,248	252,154	16,094	94%
Case 4	6,058,254	5,876,057	182,197	97%
Case 5	9,633,057	9,055,074	577,983	94%
Case 6	4.22×10^{11}	4.009×10^{11}	2.11×10^{10}	95%
Case 7	10,327,826	9m708,156	61,964	94%

because adding constraints narrows the search space, allowing the software to bypass certain exhaustive geometric comparisons. However, as constraint dimensions increase, runtime slightly rises due to the computational overhead of evaluating more rules. The GPU, benefiting from massive thread-level parallelism, effectively masks this overhead, maintaining a remarkably constant runtime of 26.2 s. In contrast, DFMAcc utilizes customized parallel interval comparators to evaluate multi-dimensional constraints simultaneously in the spatial domain. This achieves near-zero-overhead rule scaling with strict deterministic latency (~5.2 s, a 5× speedup over the GPU), confirming its commercial-grade robustness.

Table 4: Runtime under varying fuzzy constraints.

Constraints	0	1	2	3	5	8
CPU(s)	370.4	210.1	210.5	210.8	211.4	212.0
GPU	28.2	27.6	26.5	26.2	26.2	26.2
FPGA(s)	4.9	5.1	5.12	5.2	5.2	5.2

6.4 Scalability and Efficacy Analysis

The efficacy of our pattern-aware dynamic partitioning is quantified in Fig. 11(a). By adaptively balancing workload based on geometric complexity, our algorithm achieves an average 1.27× speedup over a static grid. For instance, on Case 5, dynamic partitioning reduces runtime from 397.83s to 258.59s (1.54×). On extreme-scale industrial datasets, it reduces execution time for Case 6 (1.20×) and Case 7 (1.40×). This validates its role in mitigating hotspots and ensuring a balanced data supply.

While the 32-core CPU and A100 GPU baselines exhibit severe performance saturation on large-scale designs (e.g., GPU throughput drops to 2.81 M poly/s on Case 5, and CPU stays bottlenecked below 6 M poly/s), DFMAcc's throughput scales robustly. On Case 5, DFMAcc achieves 37.25 M poly/s. Furthermore, on extreme-scale

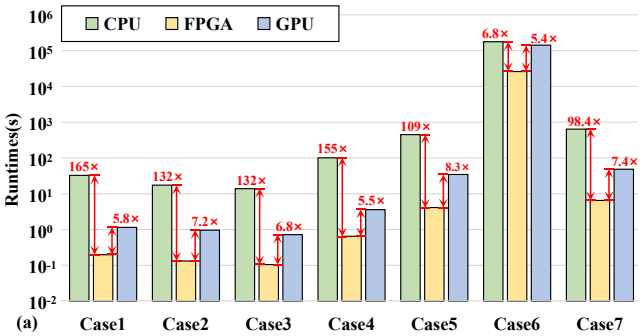


Figure 10: Hash matching engine performance.

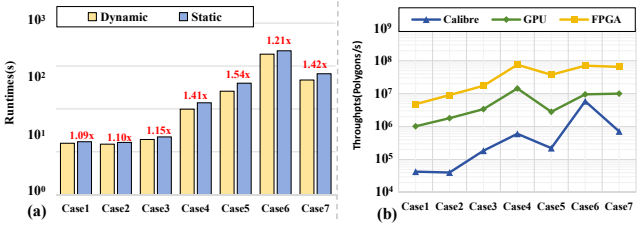


Figure 11: Partitioning efficacy and throughput performance.

industrial datasets (Cases 6 and 7), the FPGA sustains peak throughputs of ~70.50 M and 65.00 M poly/s respectively, while the GPU stalls around 9.52 M and 10.00 M poly/s. This confirms that our streaming architecture avoids the memory and control-flow bottlenecks of conventional platforms, demonstrating superior scalability for industrial workloads.

7 CONCLUSION

This paper presents **DFMAcc**, a geometry-aware streaming FPGA architecture for DFM *pattern matching*. By integrating pattern-aware dynamic partitioning with a hardware-native streaming pipeline, **DFMAcc** effectively addresses the throughput and scalability challenges of traditional CPU/GPU solutions. Comprehensive experiments demonstrate an up to 124× speedup over the industrial Calibre baseline, while maintaining 100% bit-exact accuracy. The high-throughput and scalable nature of **DFMAcc** enables more frequent DFM sign-off checks and paves the way for integrating *pattern matching* directly into the layout optimization loop. As future work, we plan to further refine the hardware acceleration logic for fuzzy matching to unlock even greater performance potentials for complex multi-dimensional DFM rules.

REFERENCES

[1] Bei Yu, Xiaoqing Xu, Subhendu Roy, Yibo Lin, Jiaojiao Ou, and David Z Pan. Design for manufacturability and reliability in extreme-scaling vlsi. *Science China Information Sciences*, 59(6):061406, 2016.

[2] R. O. Topaloglu et al. Iccad-2016 cad contest in pattern classification for integrated circuit design space analysis and benchmark suite. In *Proceedings of the IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2016.

[3] Siemens EDA. Calibre pattern matching, 2024.

[4] Semiconductor Engineering. Design rule pattern matching (knowledge center), 2024.

[5] Zhuolun He, Yuzhe Ma, and Bei Yu. X-check: Gpu-accelerated design rule checking via parallel sweepline algorithms. In *Proceedings of the IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2022.

[6] Zhuolun He, Yihang Zuo, Jiaxi Jiang, Haisheng Zheng, Yuzhe Ma, and Bei Yu. Opendrc: An efficient open-source design rule checking engine with hierarchical gpu acceleration. In *Proceedings of the ACM/IEEE Design Automation Conference (DAC)*, 2023.

[7] Jiaxi Jiang, Lancheng Zou, Wenqian Zhao, Tinghuan Chen, and Bei Yu. Pdrc: Package design rule checking via gpu-accelerated geometric intersection algorithms for non-manhattan geometry. In *Proceedings of the ACM/IEEE Design Automation Conference (DAC)*, 2024.

[8] Zhuolun He, Yihang Zuo, Jiaxi Jiang, and Bei Yu. Heterogeneous acceleration for design rule checking. In *Proceedings of the IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2023.

[9] Yibo Lin, Jiaqi Jiang, Shounak Dhar, Wei Li, Haoxing Ren, Brucek Khailany, and David Z. Pan. Dreamplace: Deep learning toolkit-enabled gpu acceleration for modern vlsi placement. In *Proceedings of the ACM/IEEE Design Automation Conference (DAC)*, 2019.

[10] X. Jiang, Y. Lin, Y. Lin, and D. Z. Pan. An fpga-accelerated maze routing kernel for vlsi designs. In *Proceedings of the 27th Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2022.

[11] Taeho Shin, Dongjun Lee, Dongwhee Kim, Gaeryun Sung, Wookjin Shin, Yunseong Jo, Hyungjoo Park, and Jaeduk Han. Laygo2: A custom layout generation engine based on dynamic templates and grids for advanced cmos technologies. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 42(12):4402–4412, 2023.

[12] Andrew Kahng, Sayak Kundu, and Dooseok Yoon. Placement tomography-based routing blockage generation for drv hotspot mitigation. In *Proceedings of the IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2024.

[13] Qianxi Chen, Yujiao Deng, Qiang Wu, and Zhixiong Di. An r-dfa-based layout pattern match method supporting fuzzy matching. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 44(10):3938–3947, 2025.

[14] Xu He, Yu Deng, Shizhe Zhou, Rui Li, Yao Wang, and Yang Guo. Lithography hotspot detection with fft-based feature extraction and imbalanced learning rate. *ACM Transactions on Design Automation of Electronic Systems*, 25(2):1–21, 2019.

[15] Po-Hsiang Tsai, Chih-Wei Hsu, Chun-Hung Lin, and Chia-Hsiang Yang. Capacitance extraction via machine learning with application to interconnect geometry exploration. In *Proceedings of the ACM/IEEE Design Automation Conference (DAC)*, 2023.

[16] Haoyang Sun, Cong Jiang, Xun Ye, Dan Feng, Benjamin Tan, Yuzhe Ma, and Kang Liu. Interpretable cnn-based lithographic hotspot detection through error marker learning. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 44(3):1031–1044, 2025.

[17] Cong Jiang, Haoyang Sun, Dan Feng, Zhiyao Xie, Benjamin Tan, and Kang Liu. Lithoexp: Explainable two-stage cnn-based lithographic hotspot detection with layout defect localization. *ACM Transactions on Design Automation of Electronic Systems*, 30(3):1–25, 2025.

[18] Silin Chen, Kangjian Di, Guohao Wang, Wenzheng Zhao, Li Du, and Ningmu Zou. Delving into topology representation for layout pattern: A novel contrastive learning framework for hotspot detection. In *Proceedings of the ACM/IEEE Design Automation Conference (DAC)*, 2025.

[19] Jea Woo Park, Robert Todd, and Xiaoyu Song. Geometric pattern match using edge driven dissected rectangles and vector space. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 35(12):2046–2055, 2016.